

2025-2027 Cloud specialist

UF: Containers-Microservices-Servlerless

Docente: Denis Maggiorotto

Il docente



Denis Maggiorotto

Denis Maggiorotto

- Managing Director and shareholder @ **Sunnyvale S.r.l.**
- 20 years of experience in ICT consulting
- Senior Software / Enterprise Architect @ Major companies in public utility, telco, TV broadcasting and banking sector
- Oracle University Principal Instructor regarding Java technologies (Micro Edition, Standard Edition and Enterprise Edition) and Oracle's middleware products.
- Independent IT professional trainer and public speaker

Denis Maggiorotto

 denis.maggiorotto@its-ictpiemonte.it

 @denismaggior8

 <https://www.linkedin.com/in/denismaggiorotto/>

 <https://github.com/denismaggior8>

Microservices



Introduzione

Microservices

L'architettura software a **microservizi** si riferisce ad una tecnica per progettare applicazioni flessibili ed altamente scalabili, scomponendo il software in servizi discreti che implementano specifiche funzioni di business.

Questi servizi possono quindi essere sviluppati, distribuiti e scalati in modo indipendente.

Microservices

Ogni microservizio comunica con altri servizi tramite API (Application Programming Interface) e protocolli standardizzati, e possono essere scritti in linguaggi differenti o con tecnologie differenti.

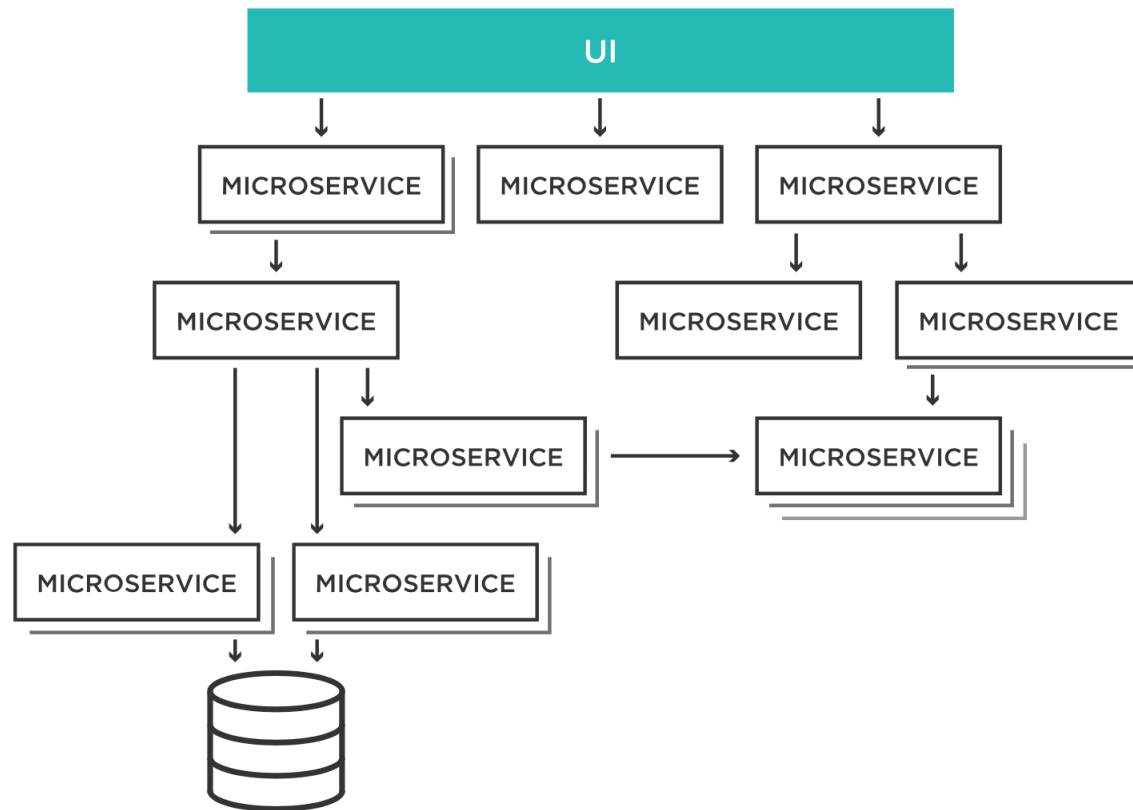
Ciò differisce completamente dai sistemi costruiti con strutture monolitiche, in cui i servizi erano inestricabilmente interconnessi, e potevano essere scalati solo insieme.

Microservices

Poiché ogni servizio ha una funzionalità ben precisa, è molto più piccolo in termini di dimensioni e complessità.

Il termine microservizio deriva da questo progetto di funzionalità discreta, non dalla sua dimensione fisica.

Microservices

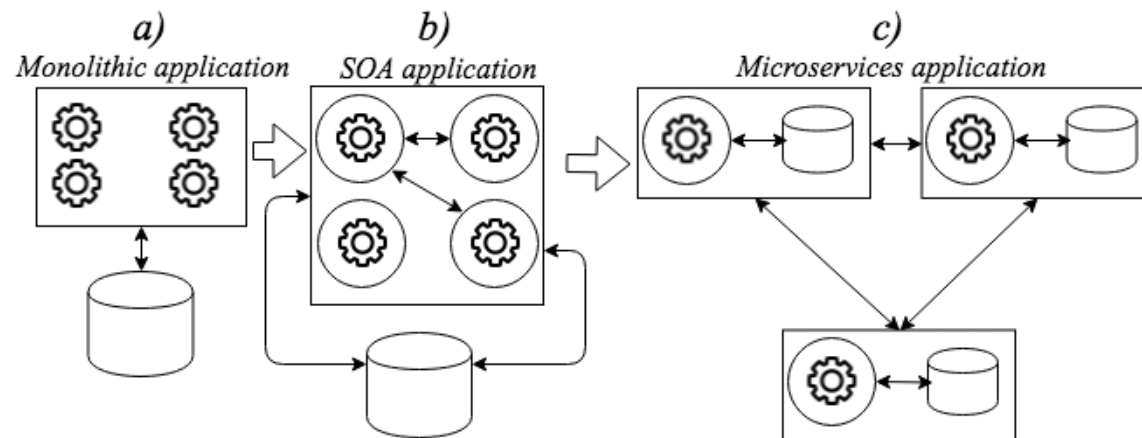


Microservices

L'architettura a microservizi arriva per risolvere le problematiche di altre due architettura molto utilizzate in precedenza (e in alcuni casi utilizzate ancora oggi):

Monolitica

SOA (Service Oriented Architecture)



Un po' di storia



Architettura monolitica

L'architettura originale, dalla quale sono derivate le altre

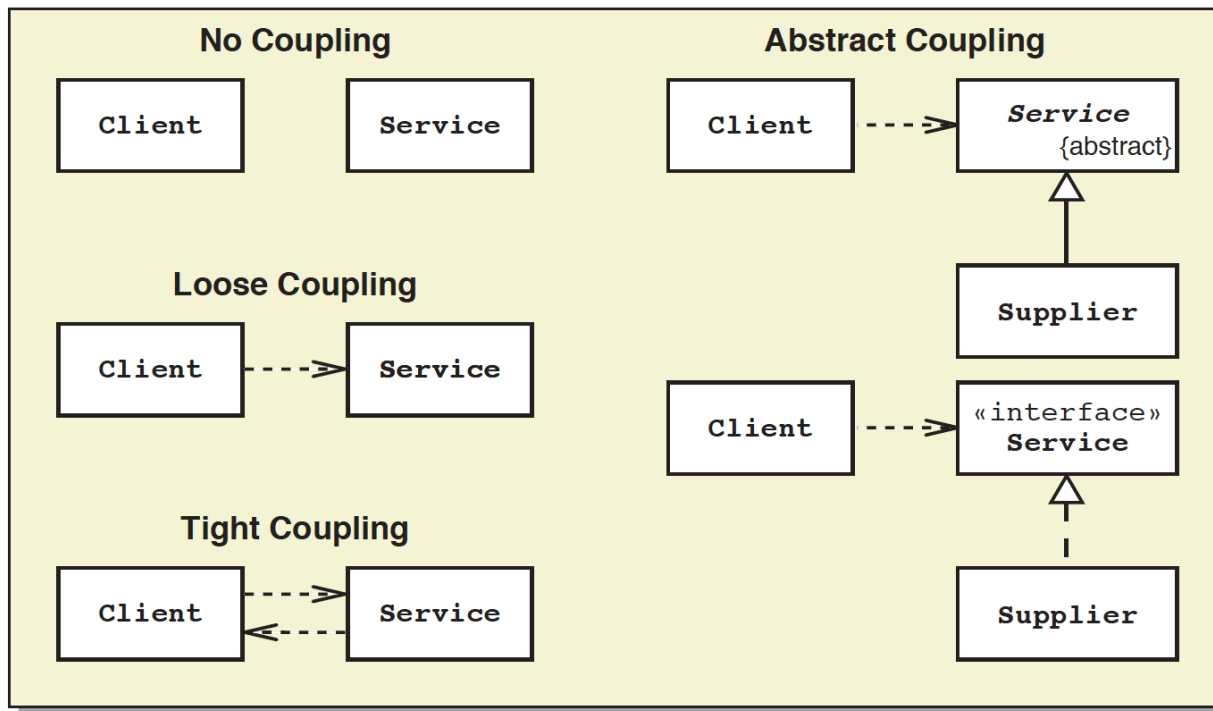
Tutte le componenti software vengono eseguite in un singolo processo

Non vi è alcun tipo di distribuzione delle sue componenti

Tutte le classi, le librerie ed altre componenti del software hanno un alto livello di dipendenza fra loro

Spesso ci si riferisce a questi tipi di software col termine «Silo»

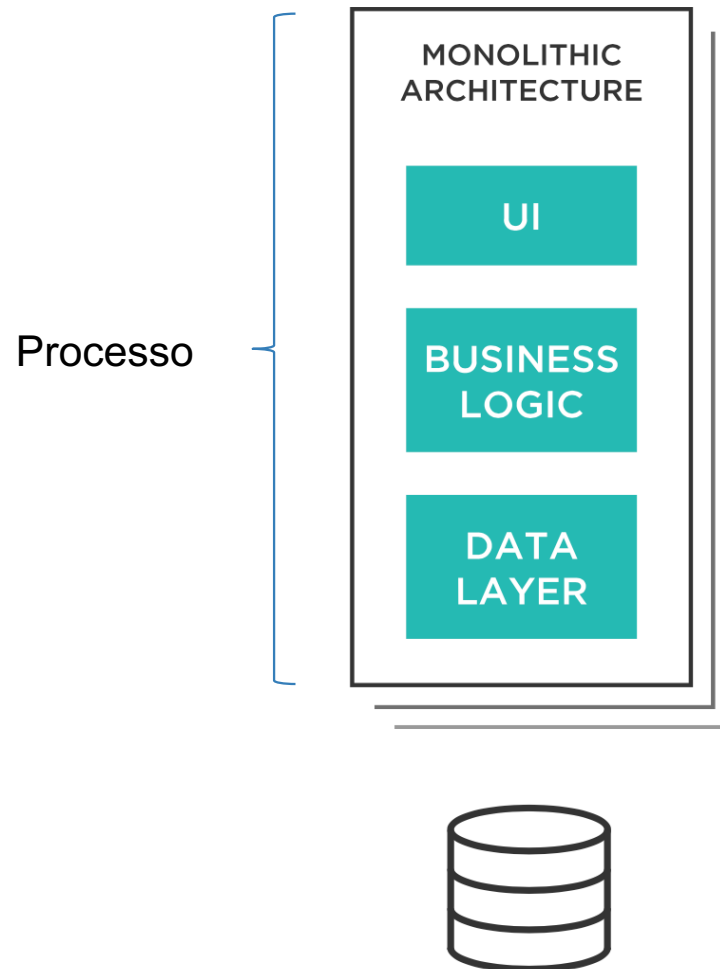
Architettura monolitica



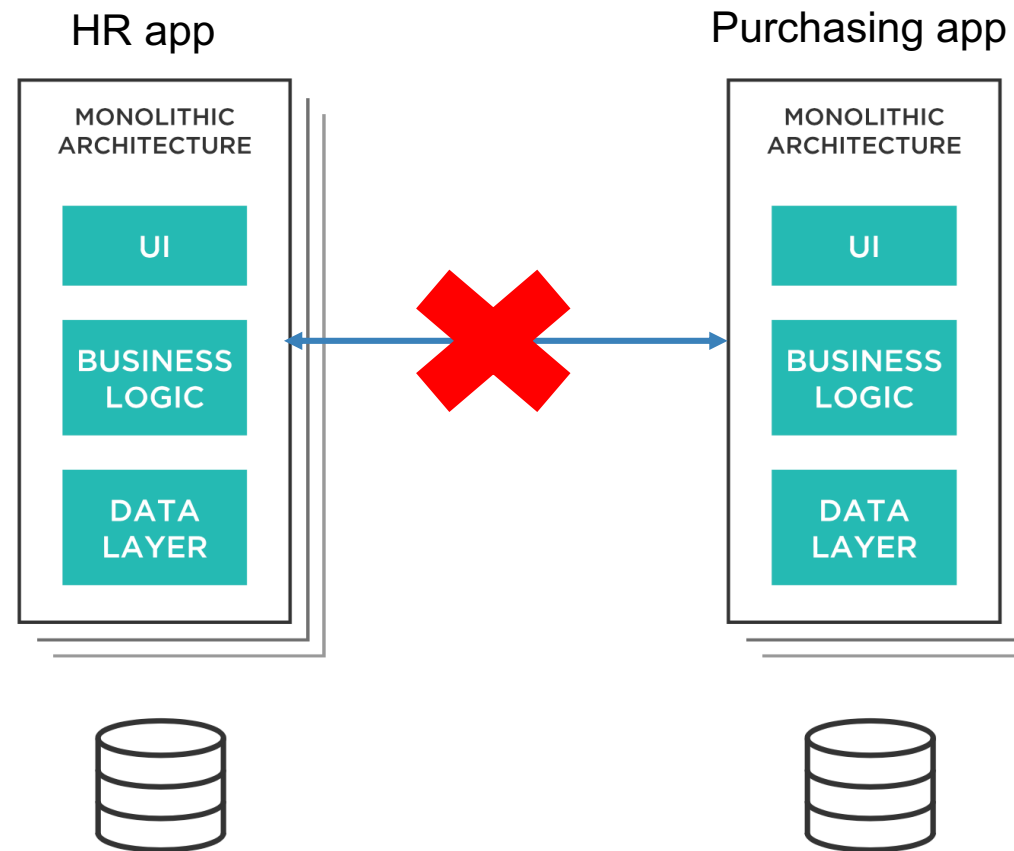
Architettura monolitica

Spesso questi software non espongono nessuna interfaccia per essere utilizzati da altri software (API)

Architettura monolitica



Architettura monolitica



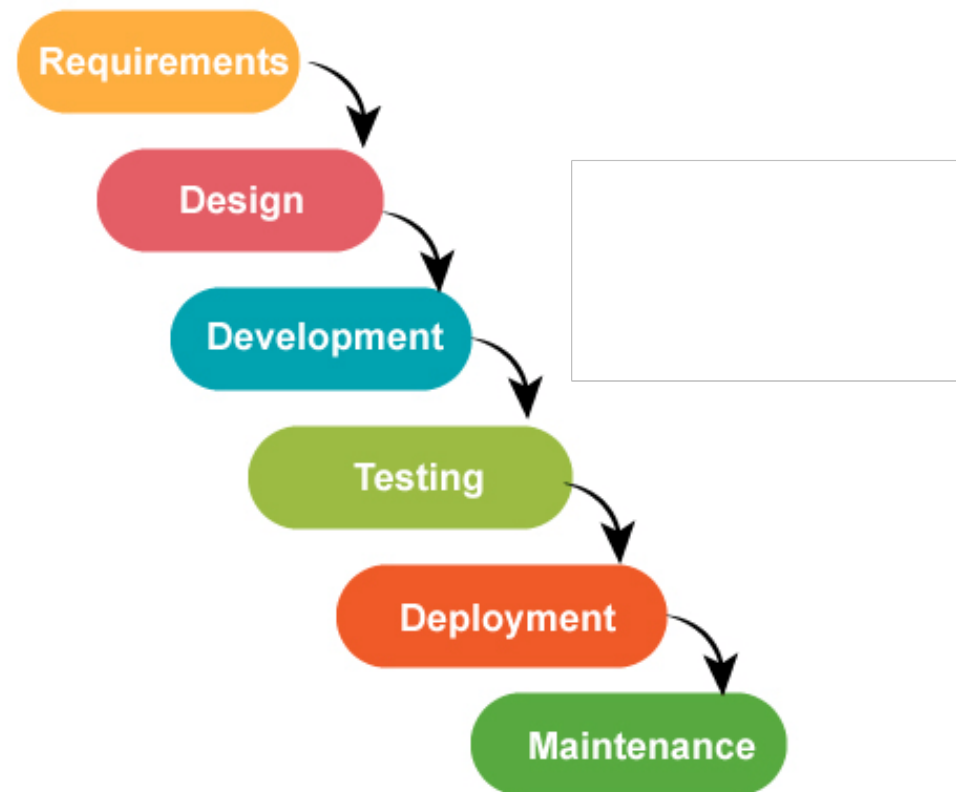
Processo di sviluppo waterfall

Il modello **waterfall** è una scomposizione delle attività di progetto in fasi sequenziali e lineari, in cui ciascuna fase dipende dai risultati conseguiti dalla precedente e costituisce il prerequisito per la fase successiva.

L'approccio è tipico del modello architetturale monolitico.

Nello sviluppo del software, tende ad essere tra gli approcci meno iterativi e flessibili, poiché il progresso scorre in gran parte in una direzione ("verso il basso" come una cascata) attraverso le fasi di ideazione, avvio, analisi, progettazione, costruzione, test, implementazione e manutenzione .

Processo di sviluppo waterfall



Vantaggi architettura monolitica

Più facile da progettare (no rete, no latenza, no protocolli di comunicazione, tutto in un singolo processo)

E' più performante (no latenze di rete, no overhead per serializzazione/deserializzazione oggetti, etc)

E' più facile da gestire (file di log centralizzati, unico processo da monitorare)

Service Oriented Architecture (SOA)

Il termine fu coniato nel 1998

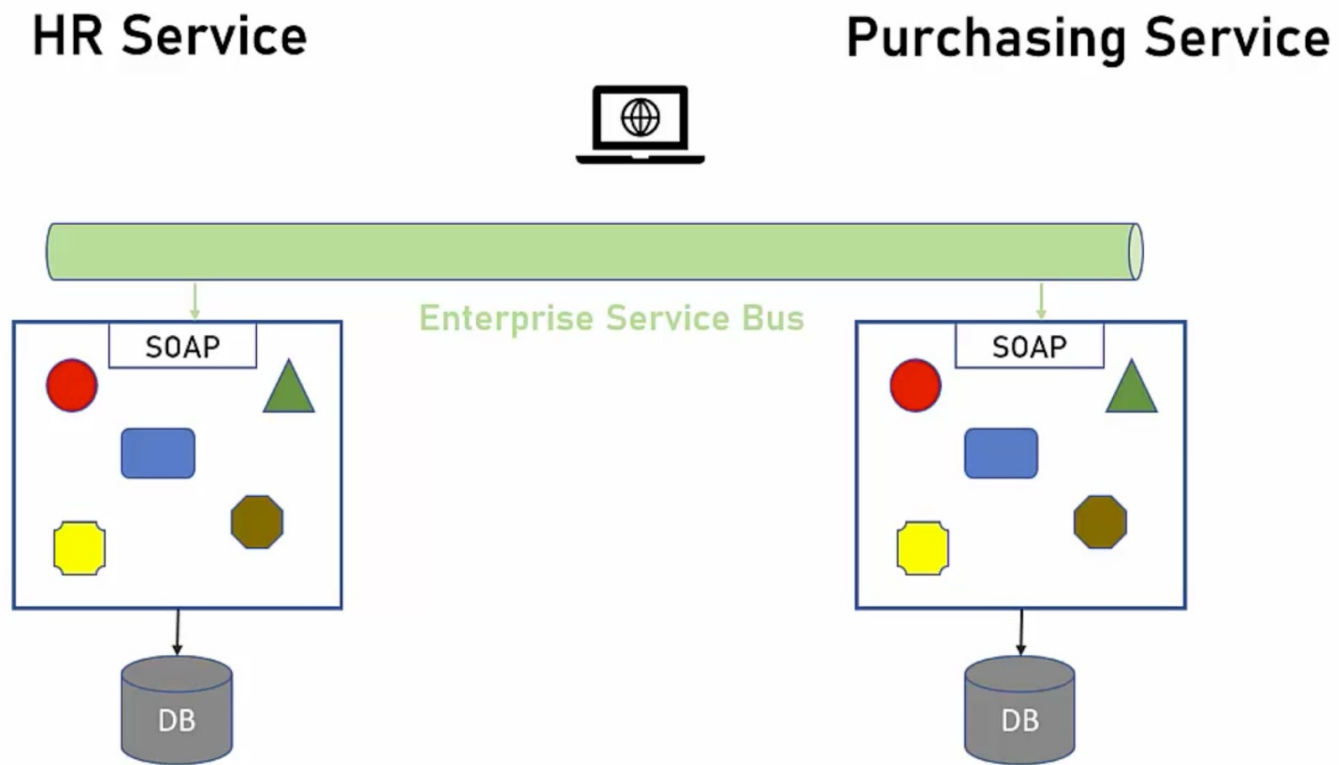
Le applicazioni sono composte da processi che espongono servizi al mondo esterno

Ogni servizio «pubblicizza» le proprie funzionalità tramite file contenenti metadati

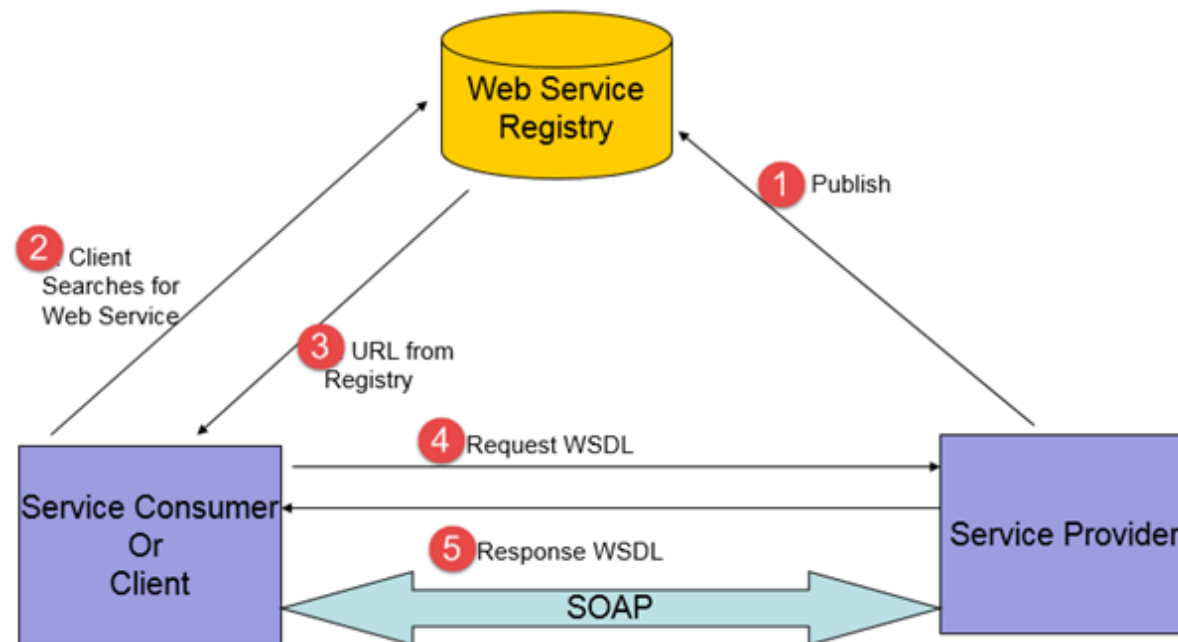
E' modello architetturale che si basa molto sugli standard XML, XML Schema, XSLT, SOAP, WSDL, etc...

Spesso viene fatto uso di Enterprise Service Bus (ESB) per mediare tra client e servizi o tra un servizio e l'altro (autenticazione, trasformazione, adattamento di protocollo)

Service Oriented Architecture (SOA)



Service Oriented Architecture (SOA)



Vantaggi architettura SOA

Dati e funzionalità vengono condivisi in modalità standard

Ogni servizio o client poteva esser scritto in linguaggi di programmazione differenti (polyglot)

Ha consentito per la prima volta la comunicazione fra prodotti/linguaggi differenti utilizzando standard (XML, SOAP)

Problemi con architetture monolitiche e SOA

Architetture monolitiche e SOA hanno mostrato molti problemi (complessità, costi, tecnologia, etc)

Molti problemi erano così rilevanti che hanno portato all'estinzione di alcuni modelli architetturali (SOA ad esempio)

Svantaggi architettura monolitica

Eseguibile (exe, jar, war, ear) di grandi dimensioni

Tempo di start/deploy molto lungo (minuti)

Poco flessibile (la modifica più banale richiede una nuova versione dell'intera applicazione)

Molti sviluppatori scrivono codice nello stesso repository (conflitti in fase di merge, necessità di coordinamento tra team di grandi dimensioni)

Scala molto difficilmente (il numero di utenti dev'essere noto in fase di progettazione e difficilmente può variare)

Svantaggi architettura monolitica

Non è adatta per internet-facing app (il numero di utenti non è noto per definizione)

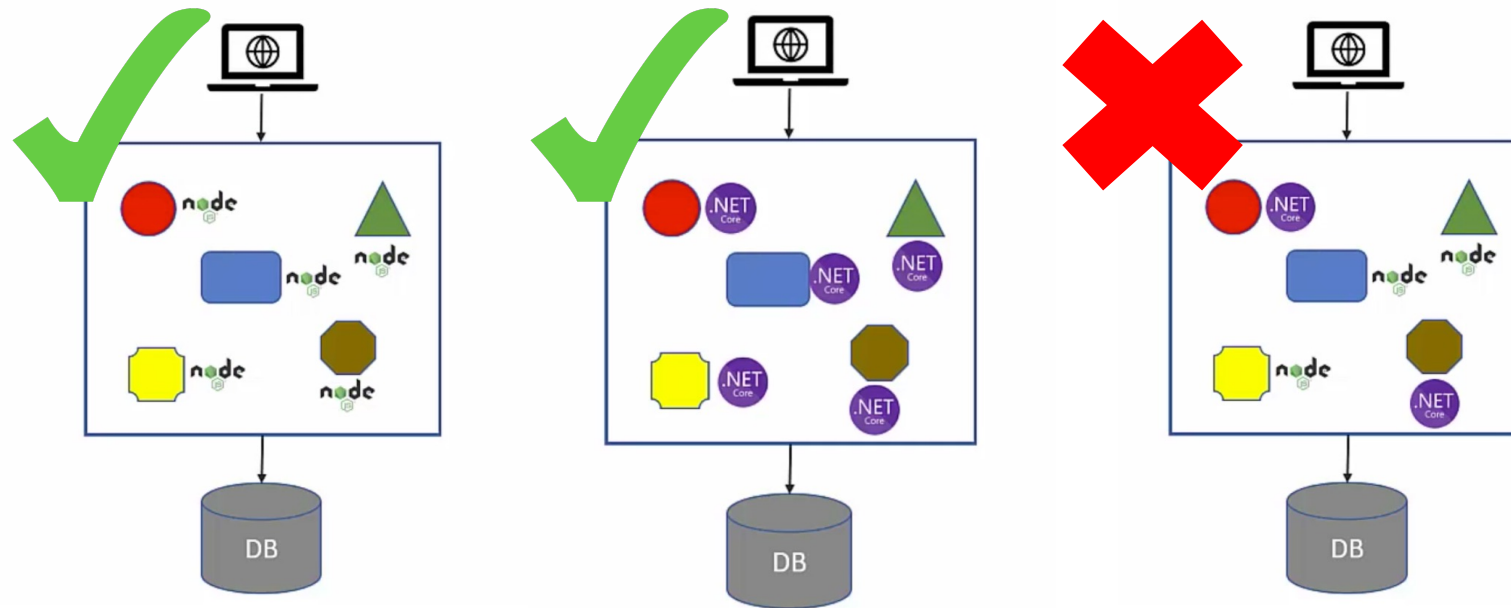
Non consente rilasci molto frequenti

L'applicazione è scritta con una singola tecnologia, cambiare tecnologia significa rifare l'applicazione (difficile sperimentare nuove tecnologie)

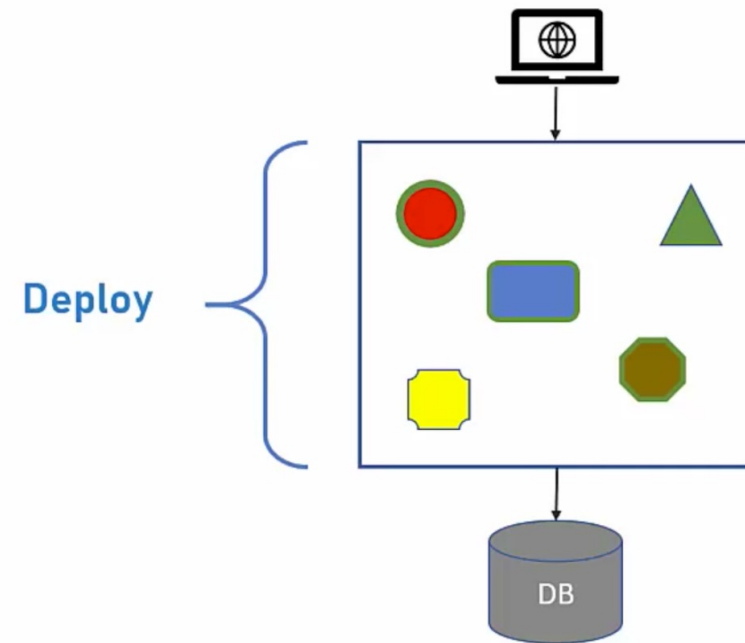
Upgrade tecnologico deve coinvolgere l'intera applicazione, non solo una singola parte

Gestione poco efficiente delle risorse hardware (CPU, RAM). Tutte le risorse vengono utilizzate da tutte le componenti, senza compartimentalizzazione

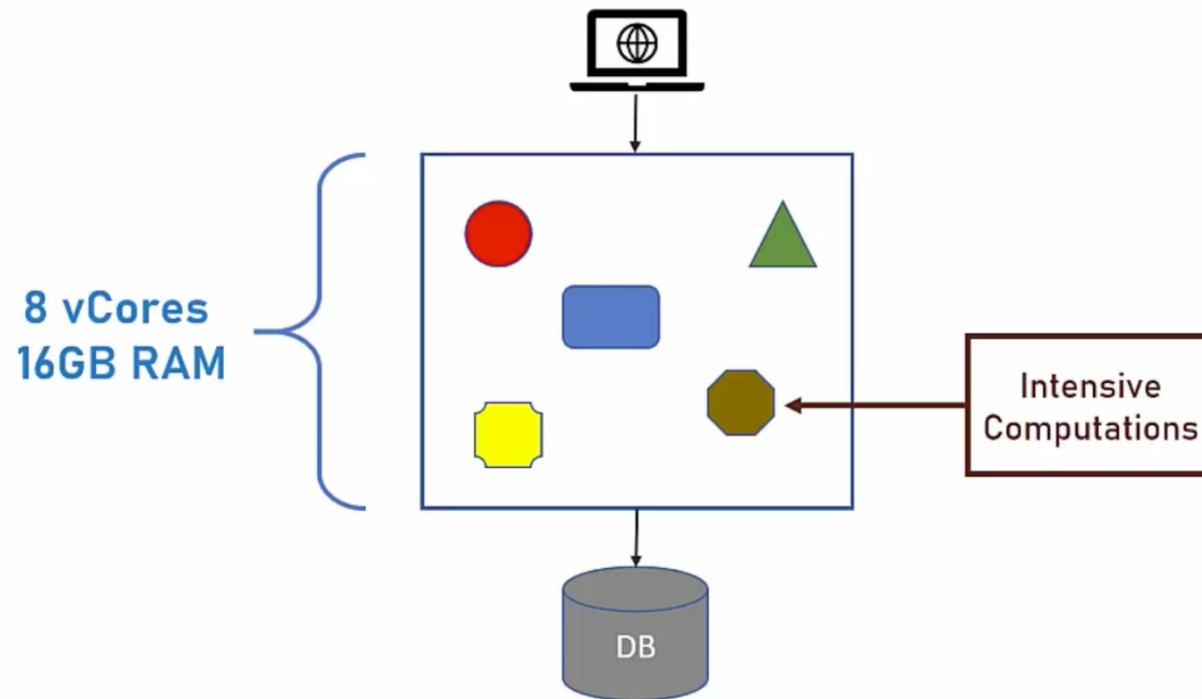
Singola architettura



Deploy poco efficienti



Uso delle risorse non compartimentalizzato



Svantaggi architettura SOA

Complessa da implementare

Molti prodotti a pagamento (vendor lock-in)

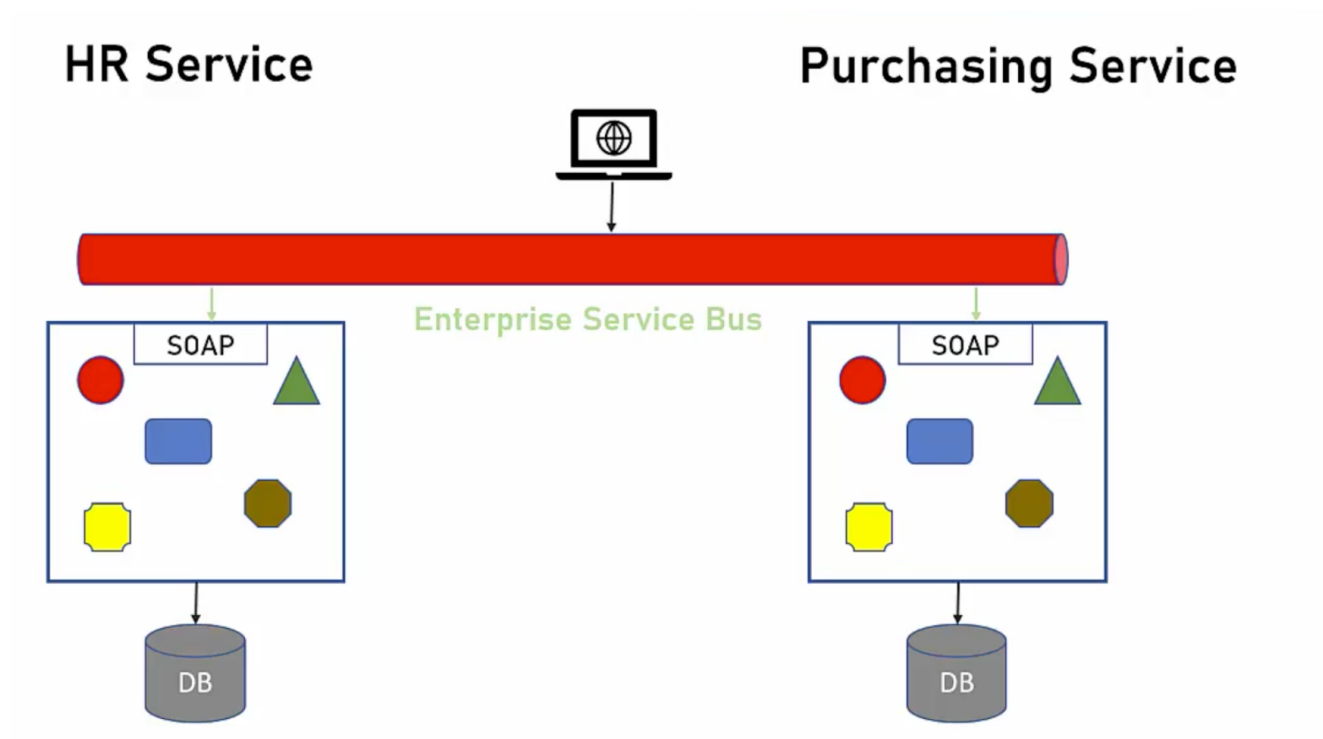
Maggior costo d'infrastruttura, di progetto e di gestione

ESB molto costoso

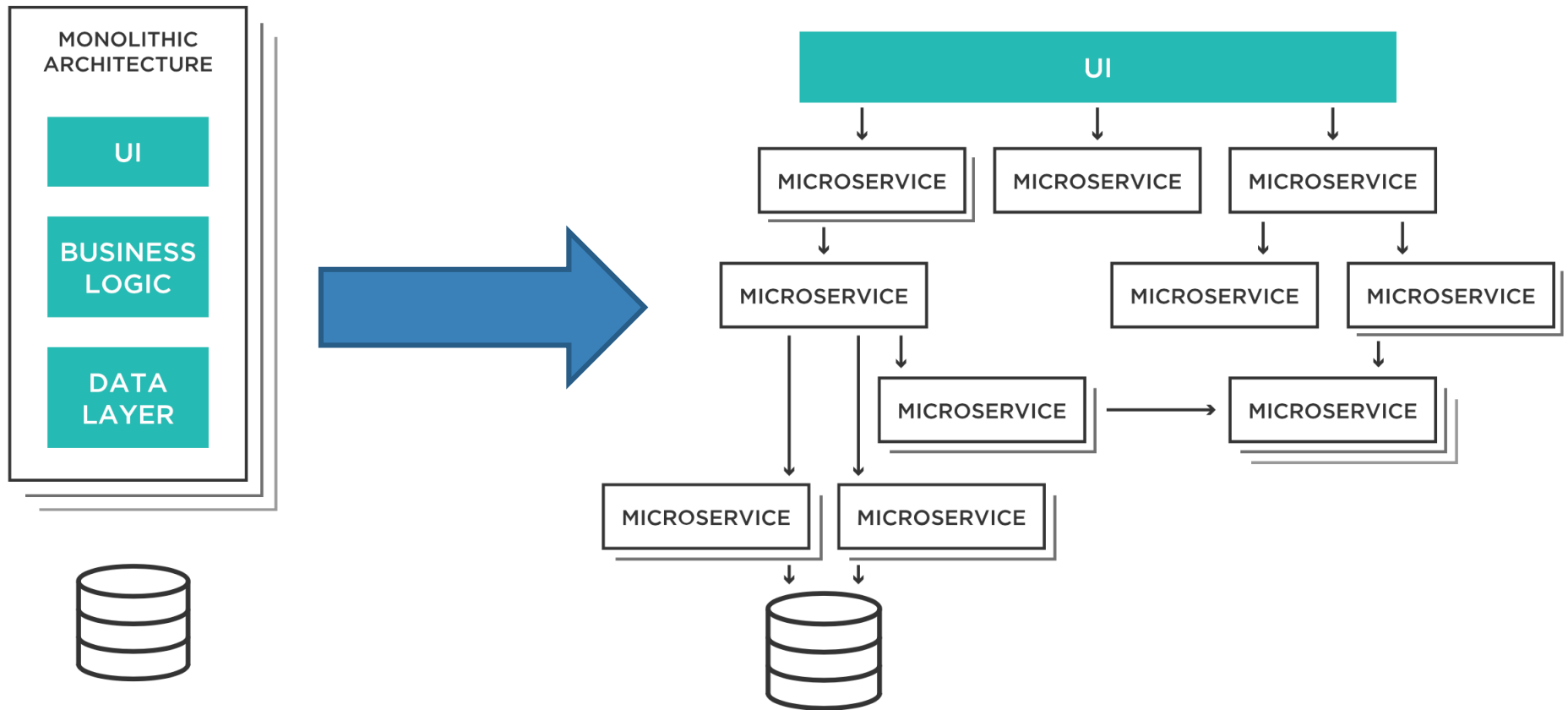
Spesso i tempi di sviluppo/gestione di architetture SOA decuplicavano rispetto a quelli necessari per architetture monolitiche

L'applicazione SOA è divisa in servizi, ma è messa in produzione in modo monolitico, questa è la principale differenza con i microservizi

ESB complesso, costoso e di difficile gestione



Da monolite a microservizi



Microservizi

Il termine microservizi viene coniato nel 2011

I problemi delle architetture monolitiche e SOA hanno ispirato il paradigma a microservizi

Con i microservizi i software sono modulari (invece che monolitici) ma con semplici API di comunicazione (REST/JSON invece di SOAP/XML)

Nel 2014 Martin Fowler e James Lewis pubblicano il loro articolo «Microservices» ed il paradigma diventa molto popolare fino ad oggi

Microservizi

<https://martinfowler.com/articles/microservices.html>

martinFowler.com

RefactoringAgileArchitectureAboutThoughtWorksRSSTwitter

Microservices

a definition of this new architectural term

The term "Microservice Architecture" has sprung up over the last few years to describe a particular way of designing software applications as suites of independently deployable services. While there is no precise definition of this architectural style, there are certain common characteristics around organization around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data.

25 March 2014



James Lewis

James Lewis is a Principal Consultant at ThoughtWorks and member of the Technology Advisory Board. James' interest in building applications out of small collaborating services stems from a background in integrating enterprise systems at scale. He's built a number of systems using microservices and has been an active participant in the growing

CONTENTS

- [Characteristics of a Microservice Architecture](#)
- [Componentization via Services](#)
- [Organized around Business Capabilities](#)
- [Products not Projects](#)
- [Smart endpoints and dumb pipes](#)
- [Decentralized Governance](#)
- [Decentralized Data Management](#)
- [Infrastructure Automation](#)
- [Design for failure](#)
- [Evolutionary Design](#)
- [Are Microservices the Future?](#)

SIDEBARS

I criteri che regolano i microservizi



Caratteristiche microservizi

Componentization via Services

Organized Around Business Capabilities

Products not Projects

Smart Endpoints and Dumb Pipes

Decentralized Governance

Decentralized Data Management

Infrastructure Automation

Design for Failure

Evolutionary Design

I criteri che regolano i microservizi



Componentization via Services

Componentization via Services

Il design modulare è sempre una buona idea

I componenti sono quelli che tutti insieme erogano la funzionalità applicativa

La modularità di solito è ottenuta tramite:

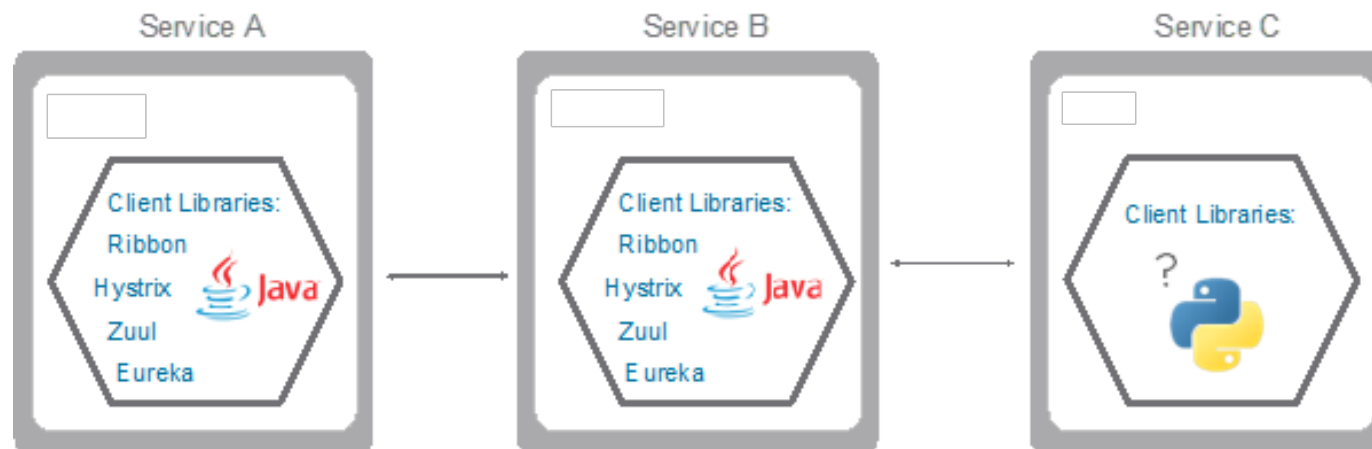
- Librerie/Moduli (richiamate direttamente dentro al processo)

- Servizi (esterni al processo, richiamati tramite protocolli)

Componentization via Services

Nel paradigma Microservizi si preferisce ottenere la modularità del software tramite Servizi

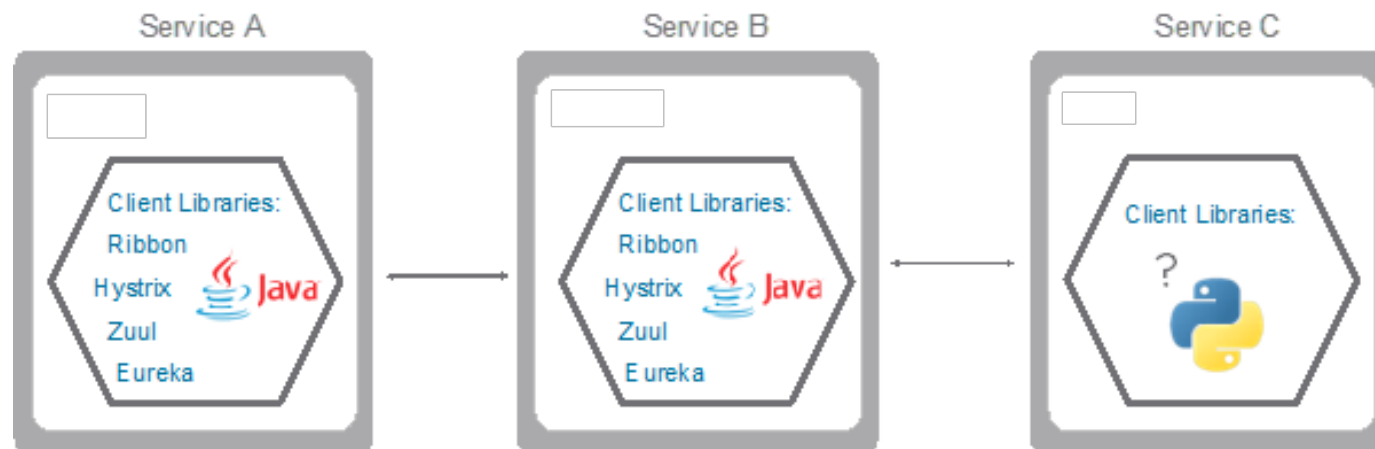
Le librerie possono essere ancora utilizzate all'interno del servizio



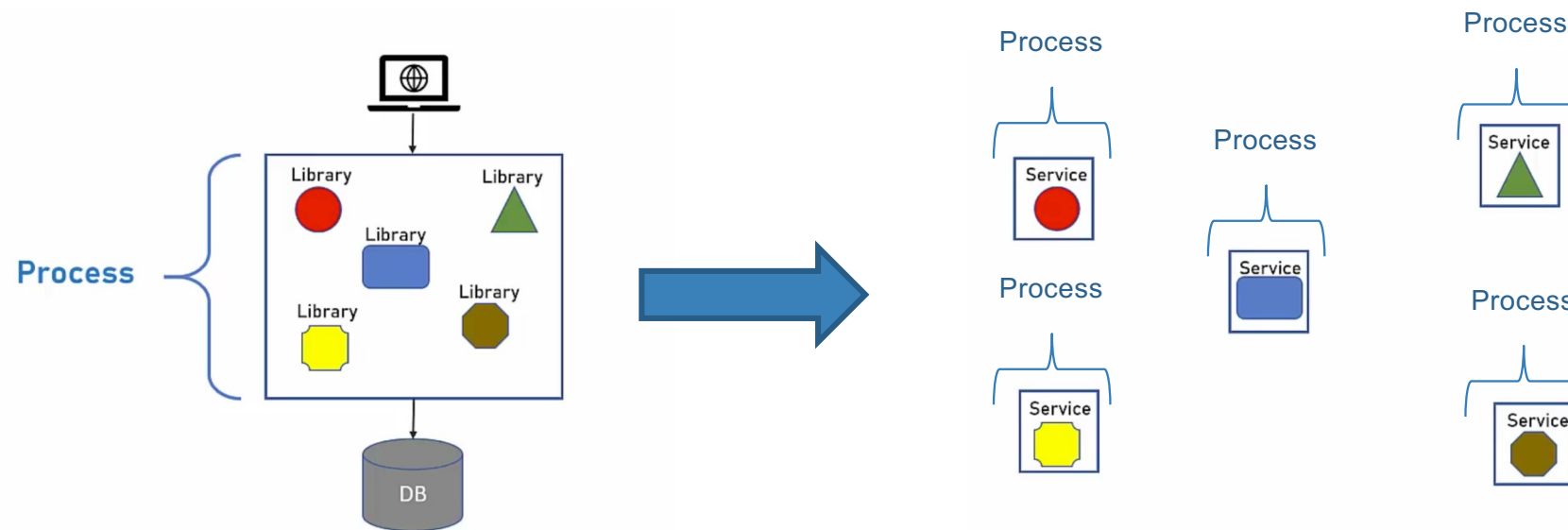
Componentization via Services

Nel paradigma Microservizi si preferisce ottenere la modularità del software tramite Servizi

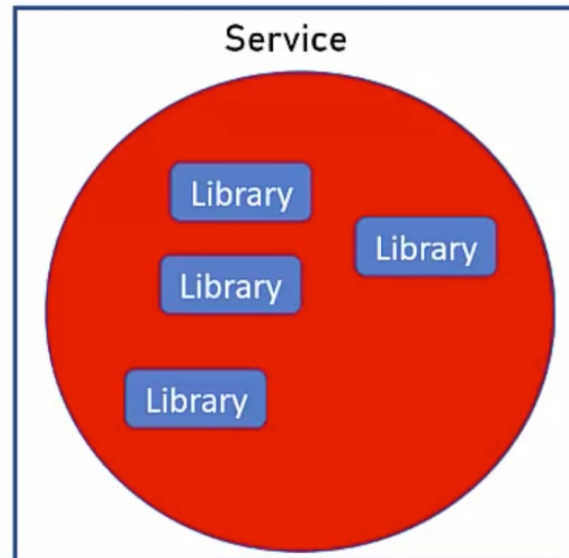
Le librerie possono essere ancora utilizzate all'interno del servizio



Componentization via Services



Componentization via Services



Componentization via Services

Perché utilizzare i servizi invece delle librerie per scomporre le applicazioni?

Ogni servizio è deployabile singolarmente (se modifico un microservizio non devo aggiornare l'intera applicazione)

Ogni servizio può esser scritto in linguaggi differenti, le librerie funzionano solo con software o altre librerie scritte nello stesso linguaggio

I Microservizi hanno un'interfaccia ben definita (Web API)

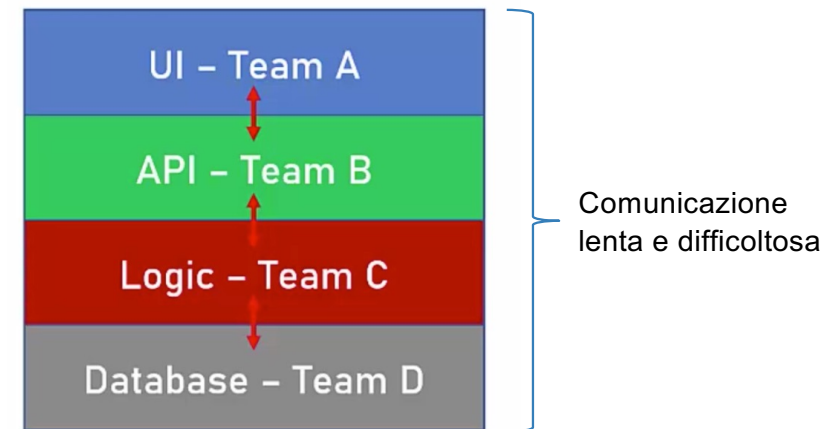
I criteri che regolano i microservizi



Organized around business capabilities

Organized around business capabilities

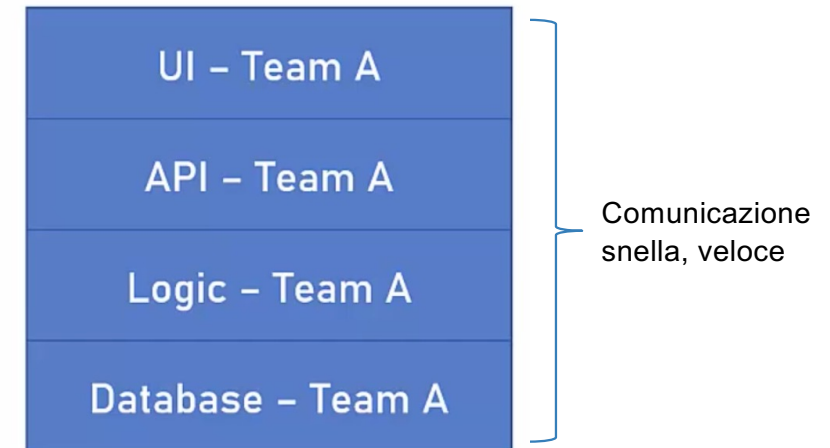
Progetti tradizionali hanno dei teams focalizzati per tecnologie/competenze (UI, API, Business logic, DB, etc)



Organized around business capabilities

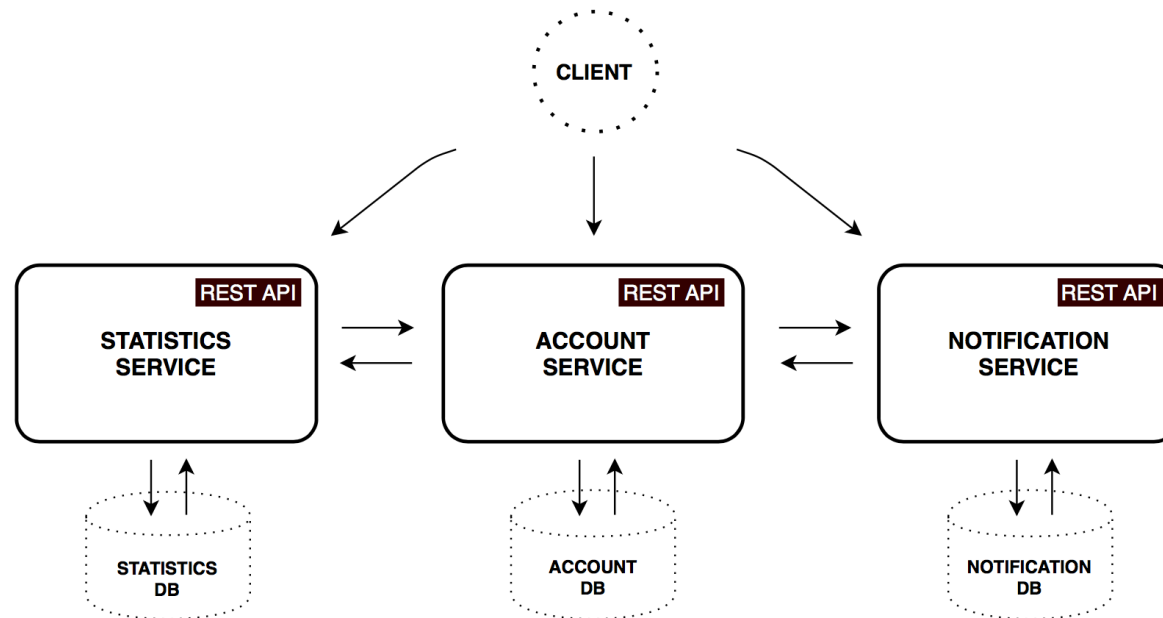
Progetti strutturati a microservizi sono organizzati con un singolo team dedicato ad un singolo microservizio.

Il Team ha un solo obiettivo, erogare il miglior microservizio possibile



Organized around business capabilities

Lo scopo del singolo microservizio è legato ad una (ed una sola) funzionalità di business (bonifico, gestione contatore, anagrafica utente)



Organized around business capabilities

Perché l'organizzazione per «business capabilities» è migliore?

Processo di sviluppo più snello ed efficace

Confini meglio definiti (definiti dal business)

I criteri che regolano i microservizi



Products not Projects

Products not Projects

Con i progetti tradizionali, l'obiettivo è quello di consegnare del codice funzionante. Questo paradigma produce i seguenti:

- Il progetto finisce col primo rilascio in produzione, in alcuni casi addirittura prima

- Quando il progetto è concluso il team si sposta a lavorare sul prossimo

- Così facendo, non si instaura una relazione duratura con il cliente e le competenze vengono disperse

- Spesso i progetti producono applicazioni eccellenti ma molto lontane al desiderata del cliente

Products not Projects

Con il paradigma a Microservizi l'obiettivo è quello di consegnare un prodotto, ne consegue che:

- Il prodotto è il vero fine, il progetto è solo il mezzo per arrivarci

- Il prodotto richiede supporto costante ed un rapporto con il cliente costante e duraturo

- Soprattutto, il team è responsabile della realizzazione del microservizio così come del supporto dopo averlo rilasciato («you build it, you run it» by Werner Vogels, AWS CTO)

Products not Projects

Le motivazioni che hanno ispirato questo principio sono:

- Avere un cliente più soddisfatto

- Cambiare la mentalità dello sviluppatore (più coinvolto in quello che sta producendo, più responsabile nella realizzazione perché un giorno dovrà mantenerlo)

I criteri che regolano i microservizi



Smart endpoints and dumb pipes

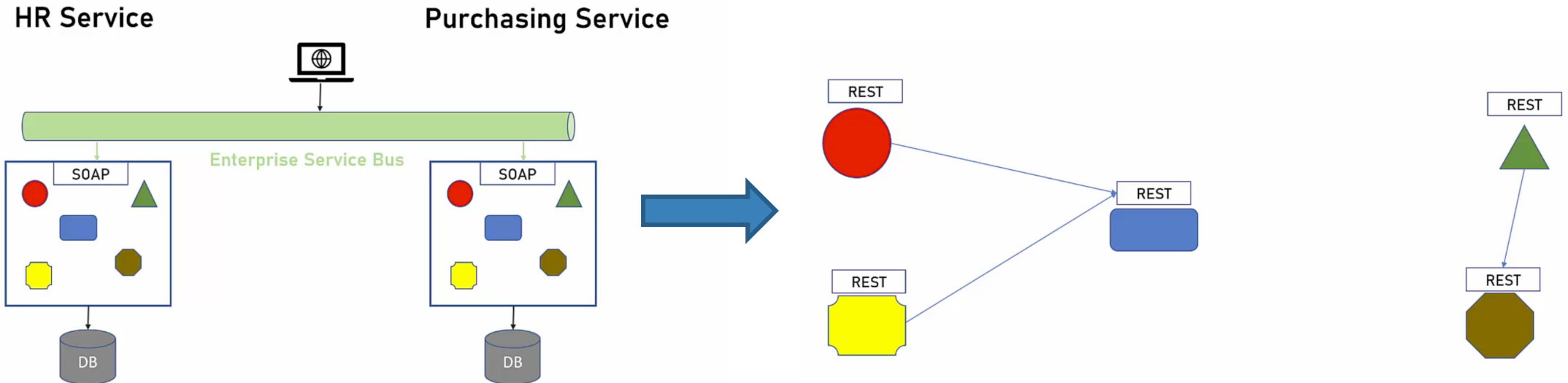
Smart endpoints and dump pipes

Questo principio ha l'obiettivo di risolvere le problematiche introdotte dalle architetture SOA, quando, sebbene le applicazioni era composte da servizi, la loro orchestrazione era complessa e governata da moltissime tecnologie complementari (SOAP, WS-*).

I Microservizi usano la forma più semplice di comunicazione, cosiddetta «dump pipes», ovvero protocolli semplici (REST/JSON invece di Web Service/XML)

I Microservizi non creano una nuova tecnologia ma utilizzano quelle già offerte dal web (HTTP invece di SOAP)

Smart endpoints and dump pipes



Smart endpoints and dump pipes

Note importanti:

La comunicazione diretta fra due servizi non è mai la soluzione migliore. Se un servizio cambiasse locazione (IP/porta) risulterebbe irraggiungibile dai client.

Per questo spesso si utilizza una componente chiamata Service Gateway.

Anche se la teoria originale incoraggiava l'uso di protocolli semplici (HTTP), negli ultimi anni sono nati altri protocolli, talvolta piuttosto complessi, per risolvere alcune inefficienze di quelli esistenti (gRPC, GraphQL, Kafka, RabbitMQ). Oggi questi protocolli fanno comunque parte del panorama tecnologico dei Microservizi.

Smart endpoints and dump pipes

Le motivazioni che hanno portato alla definizione di questo principio sono:

- Velocizzare gli sviluppi (interfacce complesse rallentano gli sviluppi).

- Migliorare la manutenibilità delle applicazioni (interfacce e protocolli complessi rendono complessa la gestione delle applicazioni).

I criteri che regolano i microservizi



Decentralized governance

Decentralized governance

Nei progetti tradizionali esiste uno standard per qualsiasi cosa

- Piattaforma di sviluppo

- Database

- Formato dei log

Gli standard vengono imposti ad ogni livello dell'applicazione, non c'è spazio per l'autonomia decisionale

Con i microservizi, ogni team è autonomo nelle decisioni che riguardano il servizio che sviluppa/gestisce

Siccome ogni team è responsabile del microservizio, prenderà decisioni migliori

Decentralized governance

Le decisioni che il team dovrà prendere per il Microservizio possono essere:

- La tecnologia di sviluppo da utilizzare (.NET, Node.js, Java, Go)

- Quali librerie utilizzare (a seconda della tecnologia possono variare)

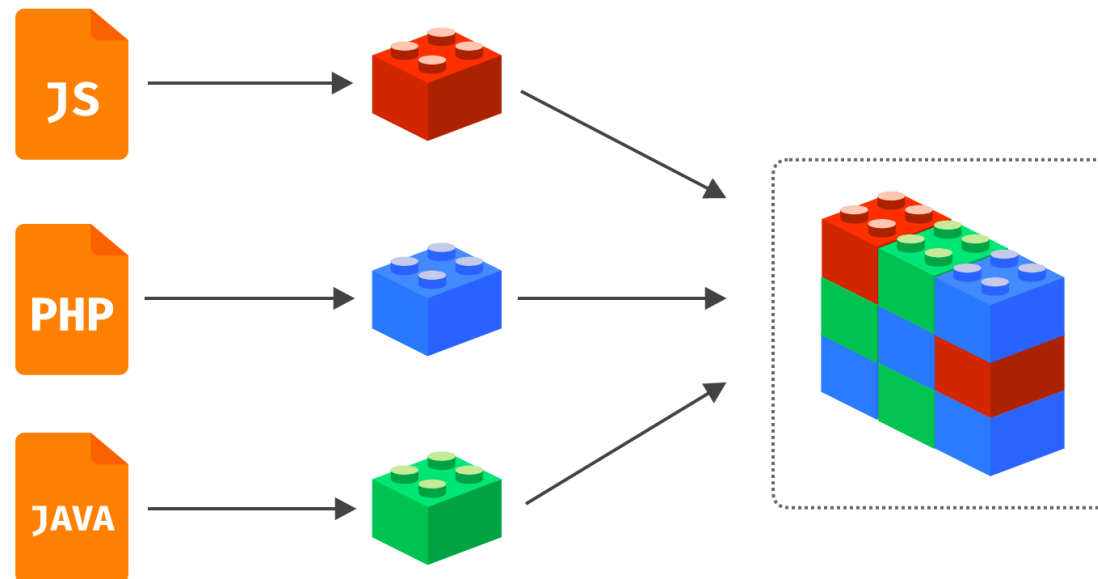
- Come gestire la produzione dei log (standard output, Kafka, ELK)

- La tipologia di database (NoSQL, RDBMS)

Tutte queste possibilità ci arrivano dalla natura dei Microservizi e dal fatto che ognuno di essi è un processo a se stante il cui scopo è quello di fornire una funzionalità tramite l'uso di Web API.

Decentralized governance

Le applicazioni composte da Microservizi sono spesso definite «multilinguaggio» (polyglot), la decisione del linguaggio da utilizzare spetta al team responsabile del microservizio senza preclusioni o scelte imposte dall'alto.



Decentralized governance

I motivi che hanno ispirato questo principio sono:

Consentire la scelta della miglior tecnologia per erogare una funzionalità (servizio)

Responsabilizzare maggiormente ogni team di sviluppo

I criteri che regolano i microservizi

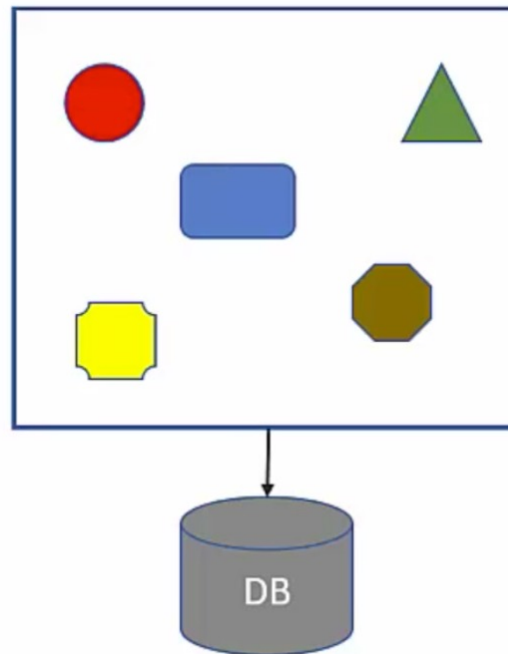


Decentralized data management

Decentralized data management

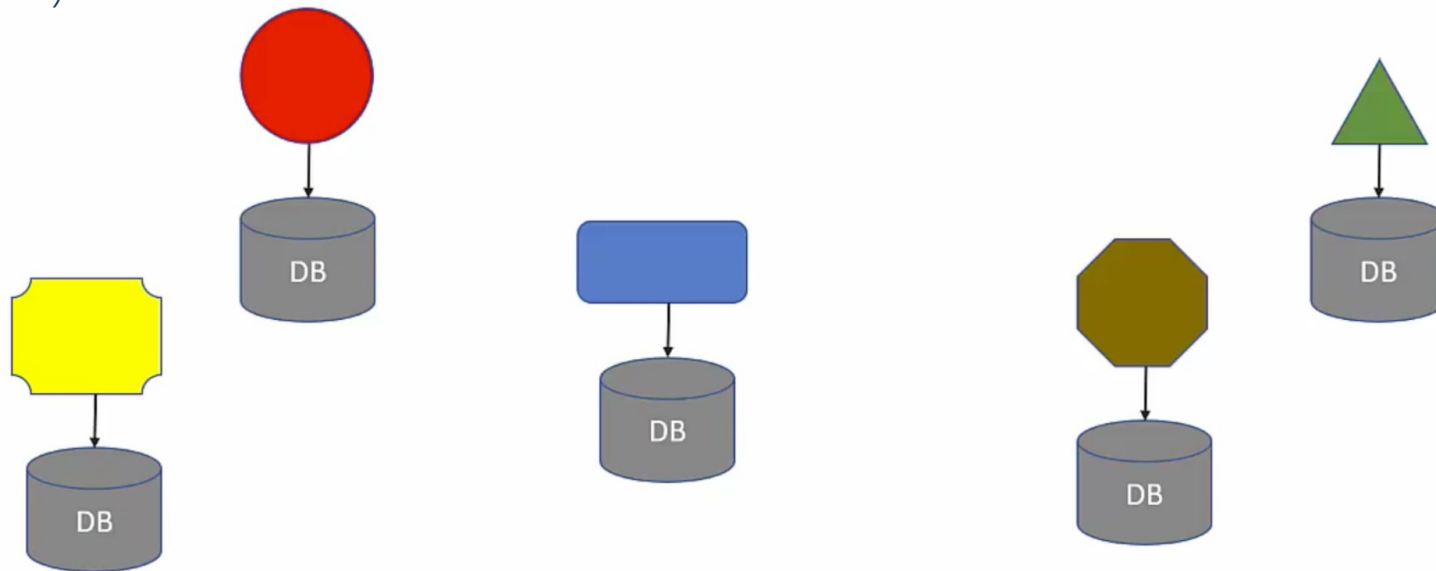
Nei sistemi tradizionali è comune immagazzinare i dati in un unico database.

Il database centralizzato immagazzina i dati di ogni componente dell'applicazione (HR, Purchasing, etc..)



Decentralized data management

Con il paradigma a Microservizi, ogni Microservizio utilizza il proprio database (non sempre relazionale)



Decentralized data management

Note importanti:

Il tema della decentralizzazione dei dati è il principio più controverso e dibattuto.

La decentralizzazione dei database pone problematiche sulle transazioni distribuite, la duplicazione dei dati, la correlazione dei dati, etc.

Non è sempre attuabile.

Generalmente, se le complessità introdotte dall'applicazione di questo principio sono tante, è meglio desistere.

Decentralized data management

Perché potrebbe essere utile applicare questo principio:

Poter usare il giusto database a seconda della tipologia di microservizio (NoSQL, RDBMS)

Separare i database incoraggia l'isolamento dei dati

I criteri che regolano i microservizi



Infrastructure automation

Infrastructure automation

Il paradigma SOA soffriva dell'assenza di strumenti (pochi, molto costosi, complessi)

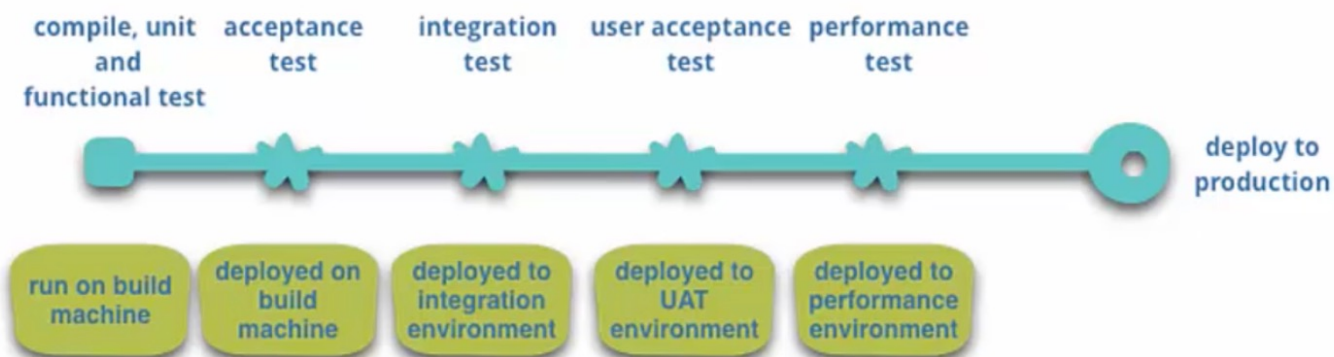
Alcuni strumenti sono molto utili nel semplificare/accelerare le operazioni di deployment:

- Test automatizzati

- Deployment automatizzato

Infrastructure automation

Esistono molte fase del ciclo di vita di un microservizio che si possono automatizzare:



Infrastructure automation

Per i Microservizi l'automazione è fondamentale in quanto nascono per poter esser rilasciati, anche in produzione, ad intervalli molto più frequenti che con le architetture SOA e quella Monolitica.

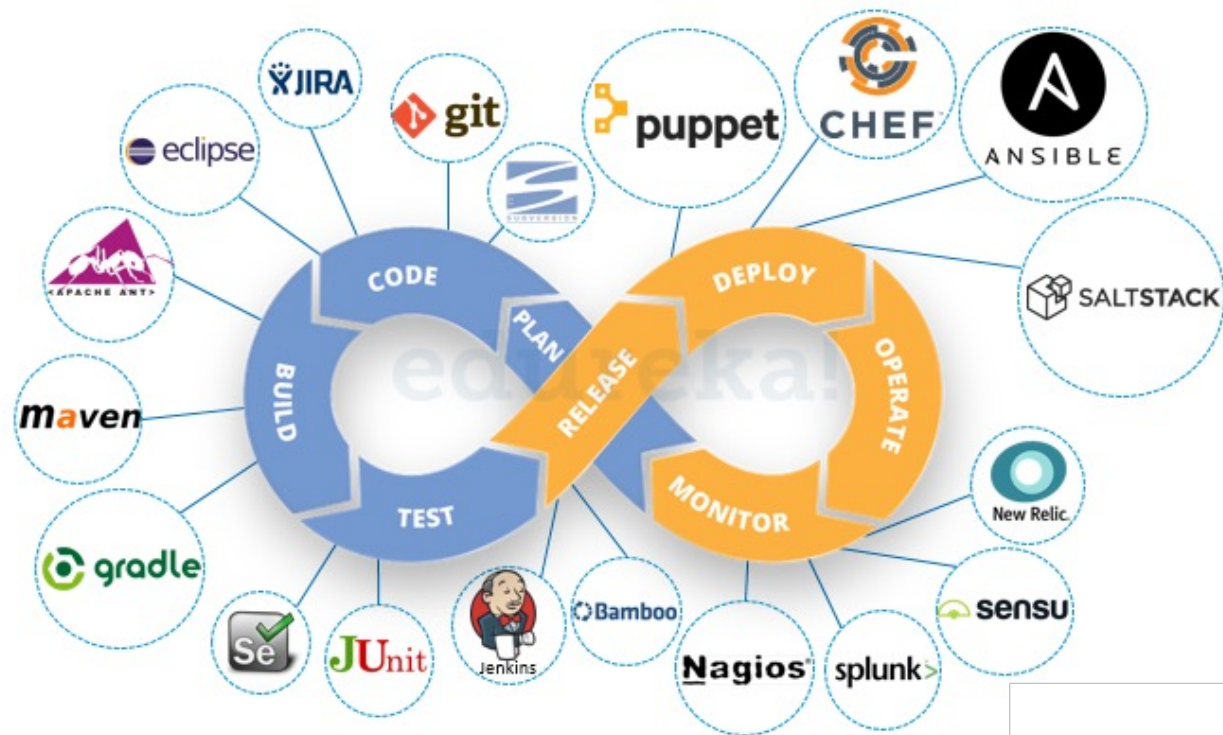
Il ciclo di vita di un Microservizio non può esser seguito manualmente.

Esistono molti strumenti per ottenere l'automazione desiderata (JUnit, Maven, GitHub Action, Jenkins, GCP Cloud Build)



Jenkins

Infrastructure automation



Infrastructure automation

Qual è la motivazione che ha ispirato questo principio?

Avere dei cicli di sviluppo/rilascio molto più frequenti.

I criteri che regolano i microservizi



Design for failure

Design for failure

In un mondo distribuito e disomogeneo, molte aspetti tendono a rendere il software passibile di malfunzionamenti:

- Tanti microservizi = elevato traffico di rete

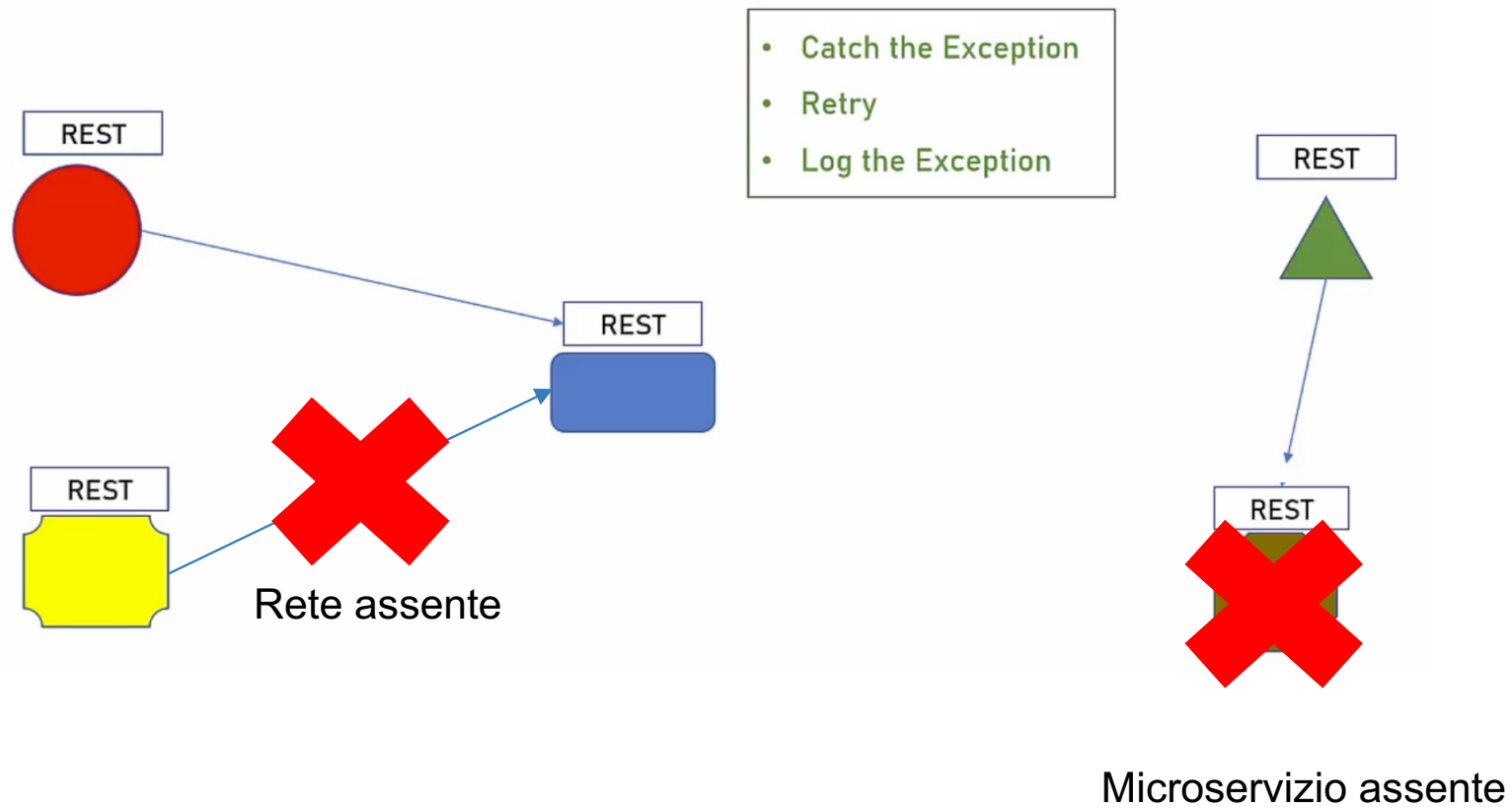
- Latenze di rete/banda non sufficienti

- Microservizio assente

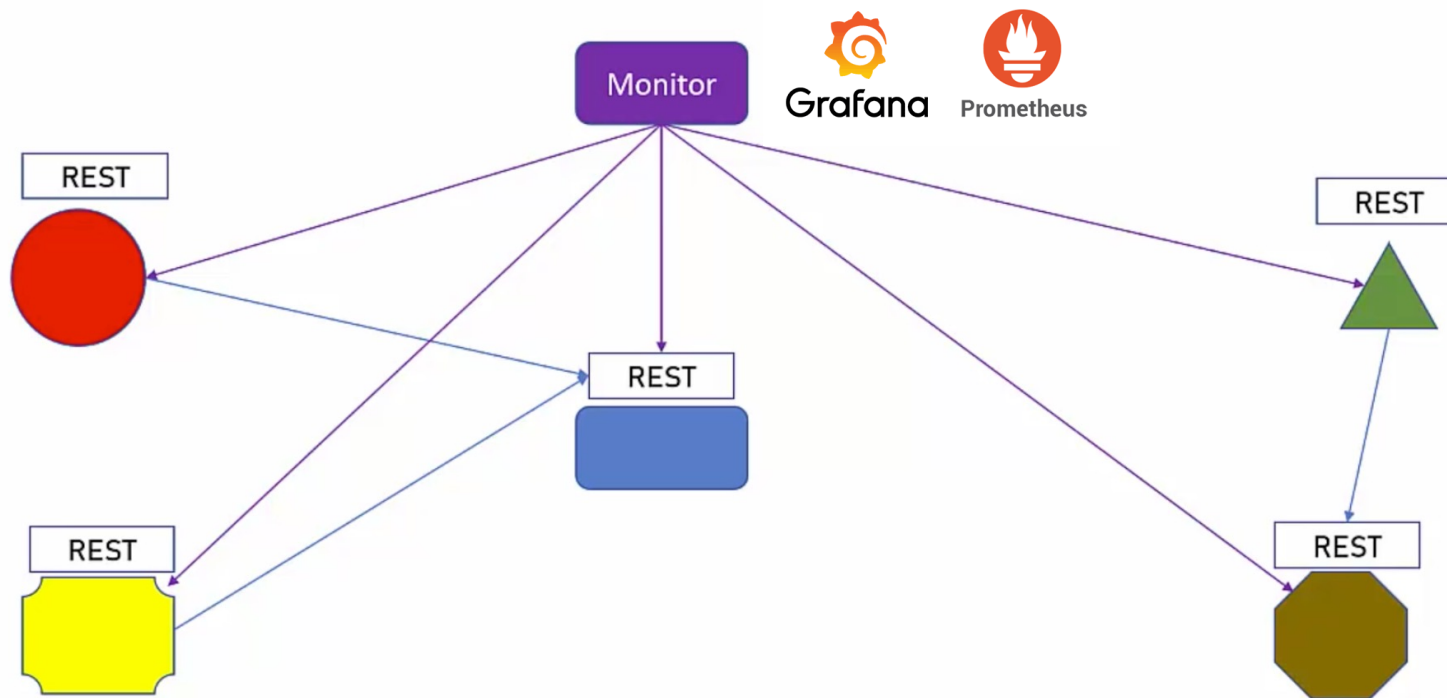
Come faccio a risolvere queste problematiche? Durante la fase di scrittura del codice è necessario essere consapevoli che i disservizi possono capitare e prendere le opportune precauzioni per gestire il disservizio nella maniera più indolore possibile.

Inoltre, una parte importante la gioca il monitoraggio e l'osservabilità di un Microservizio

Design for failure



Design for failure



Design for failure

Qual'è la motivazione che ha ispirato questo principio?

Migliorare l'affidabilità dell'applicazione e quindi la user experience

I criteri che regolano i microservizi



Evolutionary Design

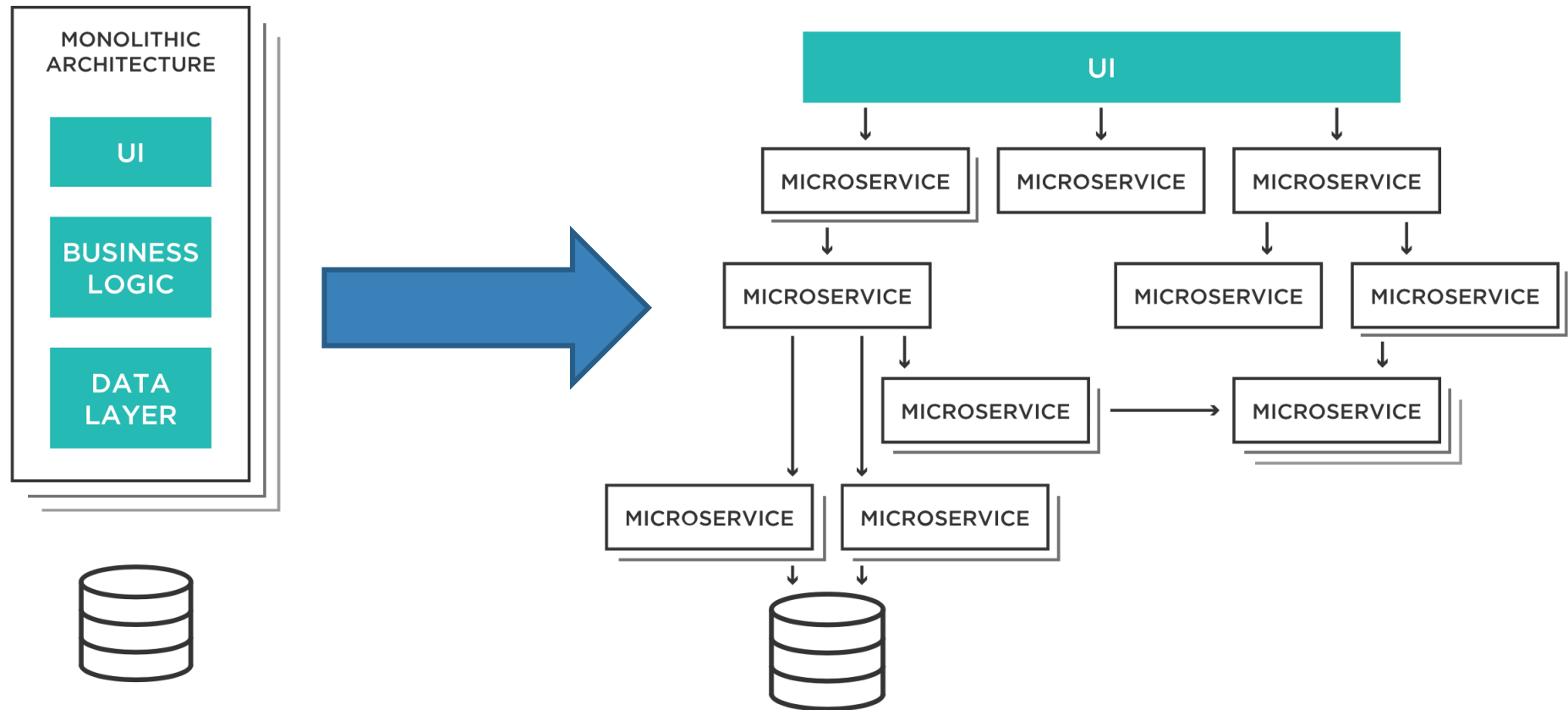
Evolutionary Design

La migrazione verso architetture a microservizi dev'essere graduale

Non c'è bisogno di rifare tutta l'applicazione da zero

Si parte in piccolo e si aggiorna ogni parte separatamente

Evolutionary Design



Design di microservizi



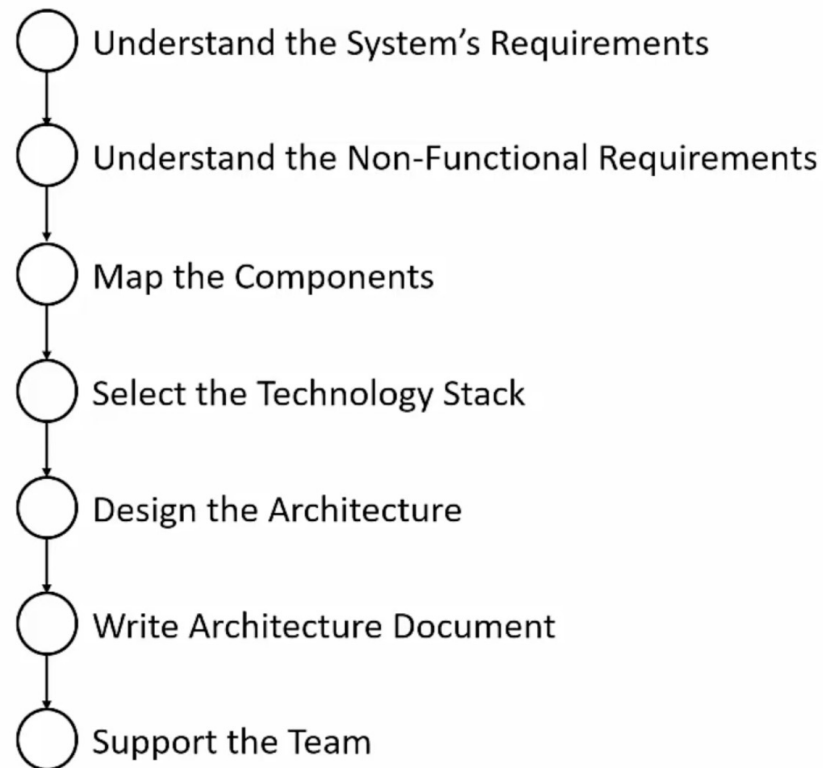
Microservices design

Per la progettazione di microservizi deve seguire un metodo ben preciso

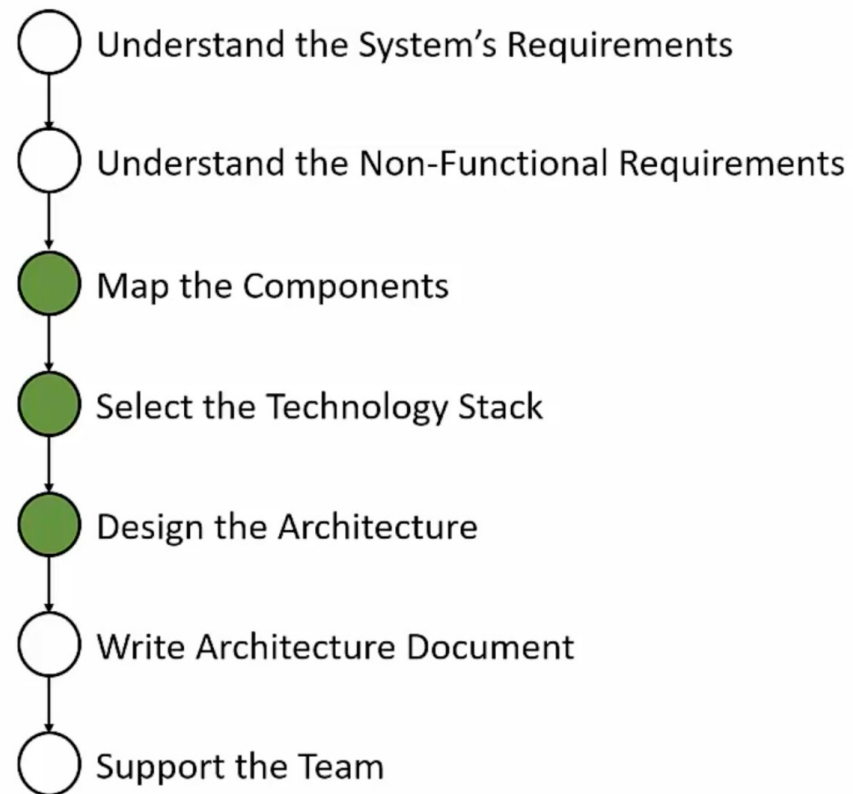
Evitare di iniziare a scrivere codice prima di aver concluso la fase di design (Plan more, code less).

La fase di design è critica per il successo dell'applicazione strutturata a microservizi (data la natura fortemente distribuita delle sue componenti)

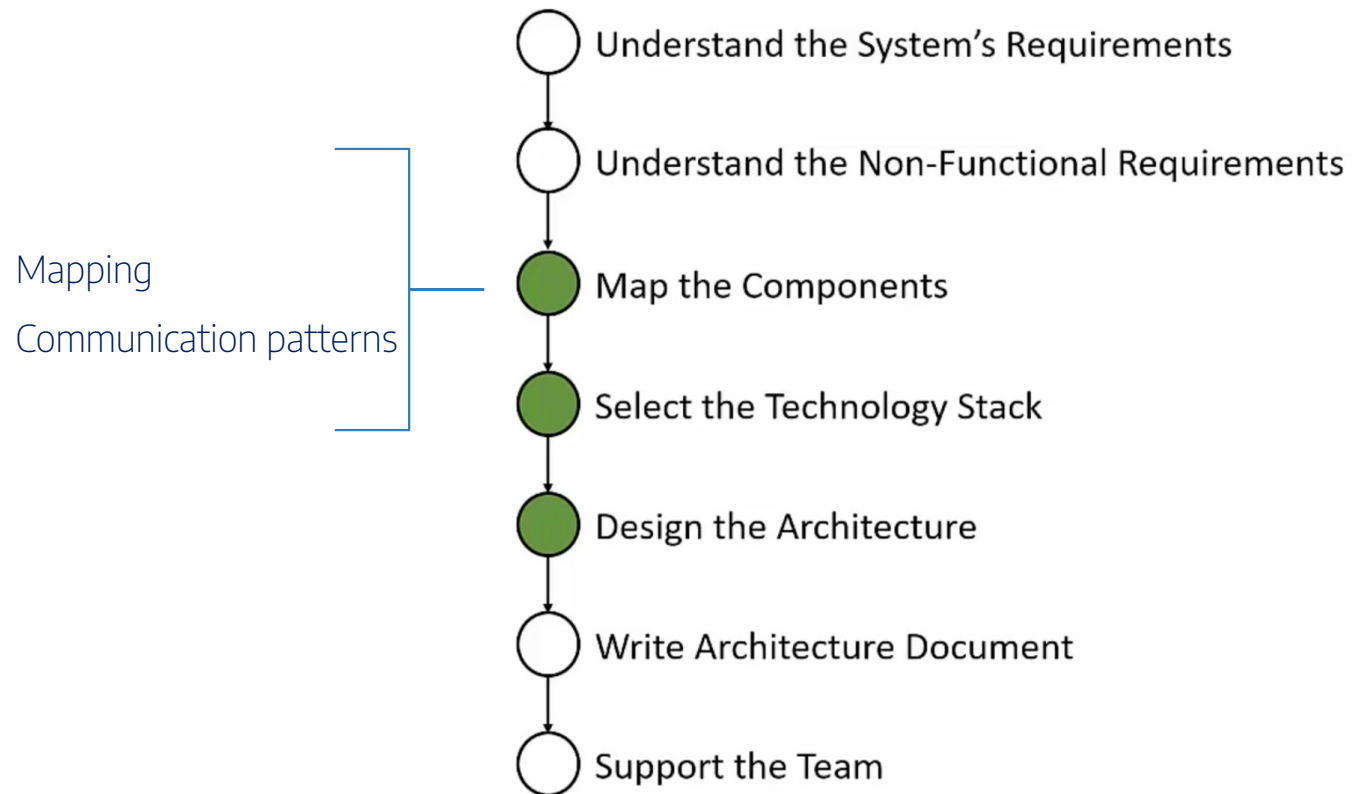
Software architecture process



Microservices focus



Microservices focus



Mapping the components

Forse il punto più importante nella costruzione del software a microservizi

Definisce la composizione dell'intera applicazione

Una volta svolta questa fase, è difficile cambiare l'architettura del software

Mapping the components

Definire le componenti (servizi) che costituiranno il sistema sulla base di:

Business requirements (requisiti funzionali)

Functional autonomy (autonomia funzionale)

Data entities (entità)

Data autonomy (autonomia dei dati)

Mapping the components

Business requirements:

L'elenco dei requisiti funzionali, costruito sui processi aziendali che l'applicazione dovrà servire

Es:

Gestione ordini (aggiungi ordine, rimuovi ordine)



Gestione anagrafiche utenti (aggiunti utente)



Mapping the components

Functional autonomy:

Il servizio non deve fornire funzionalità specifiche di altri business requirements.

Es:

Estrarre tutti gli ordini dell'ultima settimana 

Estrarre tutti gli ordini di utenti con età > 20 

Possibili deroghe a questo principio, da ridurre al minimo

Mapping the components

Data entities:

Il servizio viene associato ad una entità.

Es:

Ordini ✓
Utenti ✗

E' possibile che le entità esposte da un microservizio si riferiscano ad altre entità, ma solo tramite ID

Es: Ordine -> ID Cliente ✓

Mapping the components

Data autonomy:

I dati gestiti dal microservizio sono atomici

Il microservizio non dipende da dati forniti da altri microservizi

Es:

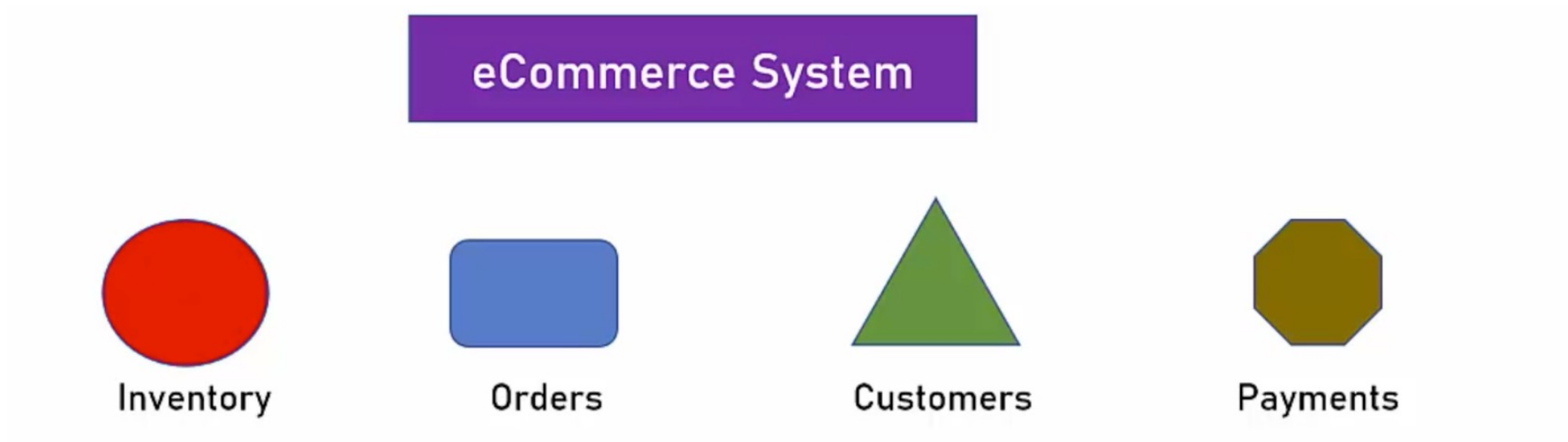
Employees service utilizza i dati di Addresses service 

Employees service include/fornisce anche l'indirizzo 

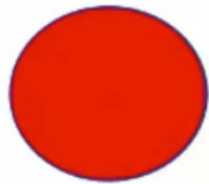


Da evitare ma l'alternativa sarebbe peggio

Mapping the components



Mapping the components



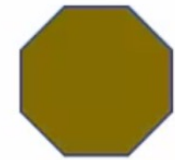
Inventory



Orders



Customers



Payments

Business Requirements	Manage inventory items	Manage orders	Manage customers	Perform payments
Functional	Add, remove, update, quantity	Add, cancel, calculate sum	Add, update, remove, get account details	Perform payments
Data Entities	Items	Orders, shipping address	Customer, address, contact details	Payment history
Data Autonomy	None	Related to Items by ID Related to Customer by ID	Related to Orders by ID	None

Mapping the components

Eccezione 1:

Fornire tutti gli utenti residenti a NY con il numero di ordini effettuati da ciascuno

Customer name	No. of orders
David Smith	16
Diane Rice	23
George Murray	22

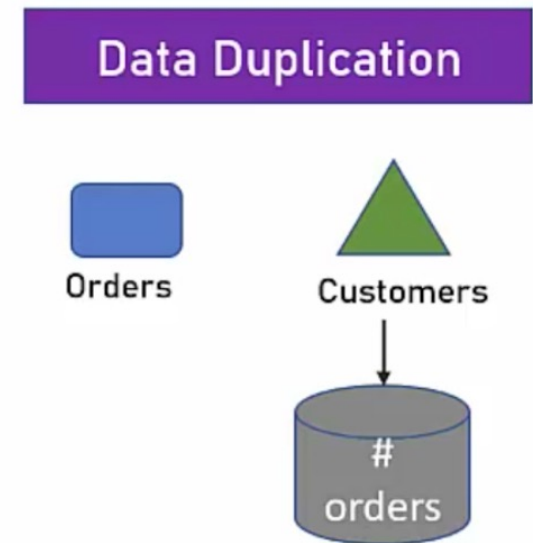
Mapping the components

Eccezione 1:

Soluzione 1: Il numero di ordini è immagazzinato nel DB dei customer.

Il numero di ordini può non esser sempre sincronizzato

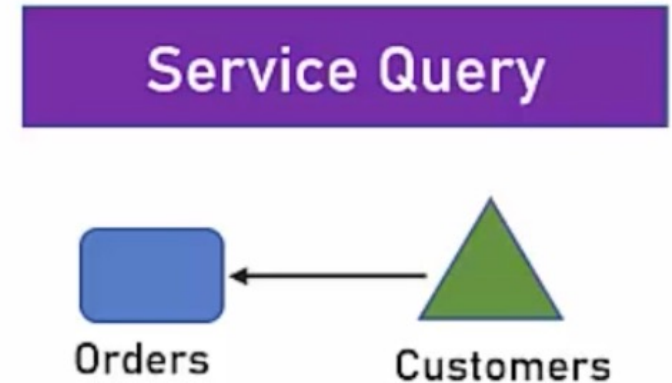
L'operazione di aggiornamento è semplice



Mapping the components

Eccezione 1:

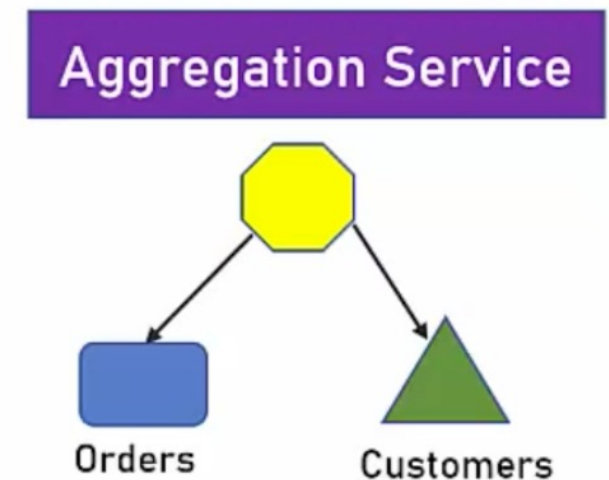
Soluzione 2: Il microservizio dei customers interroga il microservizio degli orders



Mapping the components

Eccezione 1:

Soluzione 3: Viene creato un microservizio a cappello che interroga gli altri due ed aggrega le informazioni



Mapping the components

Eccezione 2:

Fornire tutti gli ordini presenti a sistema.



Grosso volume di dati. Restituire una mole così grande di dati non è gestibile da un microservizio.

Mapping the components

Eccezione 2:

Soluzione 1: Vagliare la reale necessità di ottenere una mole così grande di informazioni.

Se serve per fare reporting, utilizzare uno strumento che acceda direttamente al DB

Mapping the components

Cross-cutting services: servizi di utilità, trasversali a tutta l'applicazione (autenticazione, autorizzazione, logging, caching)

DEVONO essere inclusi nella fase di mapping

Communication patterns

Una comunicazione efficiente tra microservizi client è fondamentale

E' importante scegliere il pattern di comunicazione migliore a seconda della necessità